

HYDRA ETL / WHITE PAPER

HYDRA DSL

VERSION 1.0

A reference guide to keywords, workflows, jobs, actions, parameters and secrets

Compatible with Hydra 0.16.1

hydraetl.com | github.com/bejaouibechir/Hydra

Contents

01 Workflows

02 Jobs

03 Actions

04 Parameters and secrets

About this edition

This publication presents the Hydra DSL version 1.0 keyword reference, with configuration examples and explanations of each keyword’s role.

Examples are shown in syntax-highlighted code panels to improve readability and reuse.

1. Workflows

Files: workflows/<name>.yaml and workflows/thread.yaml.

1.1 The workflow

version: DSL format version

```
version: "1.0"      # currently always "1.0"
```

Role. allows the engine to validate the file against the appropriate grammar.

workflow: workflow root

```
workflow:           # all workflow content sits under this key
  name: daily_sales
```

Role. declares a sequence of steps (jobs, actions and projects).

name: workflow name (required)

```
workflow:
  name: daily_sales     # identifier in logs and undo file
```

Role. identifies the workflow; also used by hdrctl workflow undo.

description: free-text description

```
description: "Load today's sales"
```

Role. documentation only; does not affect execution.

trigger: trigger (default: manual)

```
trigger: manual        # manual | schedule | webhook
```

Role. indicates when the workflow starts.

cron: schedule (required with trigger: schedule)

```
trigger: schedule
cron: "0 6 * * *"      # every day at 06:00
```

Role. five-field cron expression specifying the start time.

rollback: rollback on failure (default: false)

```
rollback: true         # undo completed steps when one fails
```

Role. triggers undo operations for completed steps in reverse order.

steps: list of steps (required)

```
steps:
  - name: load
    type: job
    job: ../jobs/orders
```

Role. contains the steps executed by the workflow.

1.2 The step

name: unique step name (required)

```
- name: load_orders     # referenced in depends_on
```

Role. identifies the step and allows other steps to depend on it.

type: step type (required)

```
type: job             # job | action | project
```

Role. specifies whether the step runs a job, an action or another project (section 1.4).

job: job path (with type: job)

```
type: job
job: ../jobs/orders     # directory containing sources/destinations/pipeline
```

Role. points to the job directory to execute.

action: action name (with type: action)

```
type: action
action: log             # one of 11 actions (see section 3)
```

Role. selects the action to execute.

params: action parameters

```
action: log
params:
  message: "Start loading"
```

Role. provides the action inputs; supports {{ param: }} and ${SECRET:}. With type: project, the parameters passed to the called project.

depends_on: dependencies (default: [])

```
- name: publish
  depends_on: [load, clean]     # starts after load AND clean finish
```

Role. builds the execution graph; independent steps can run in parallel.

on_failure: failure behavior (default: fail)

```
on_failure: continue     # fail | skip | continue
```

Role. fail stops the workflow; skip omits dependent steps; continue still executes them.

enabled: enables/disables the step (default: true)

```
enabled: false    # skip the step without deleting it
```

Role. temporarily disables the step.

when: execution condition

```
when: "{{ param:publish_enabled }} == True"      # run only if true
```

Role. runs the step according to a safe expression (parameters or another step's result).

retry: retry policy

```
retry:
  max: 3               # retries after the first failure
  delay: 10            # seconds to wait
  backoff: exponential      # fixed | exponential (delay doubled)
```

Role. handles transient failures (network or database unavailability).

undo: action undo procedure

```
type: action
action: bash
params: { command: "touch /tmp/flag" }
undo:
  action: bash
  params: { command: "rm /tmp/flag" }      # executed when rollback: true
```

Role. required for webhook, email, bash, powershell, ssh and python (or use undo: none). A job is undone automatically; a project step cannot be undone, so rollback: true refuses it.

1.3 The thread

thread: root of thread.yaml

```
thread:               # list of stages
  - mode: sequential
    workflows:
      - name: load_referentials
```

Role. sets the execution order of all project workflows (hdrctl run .).

mode: stage execution mode (default: sequential)

```
mode: parallel        # sequential | parallel
```

Role. runs workflows within a stage sequentially or in parallel.

delay_after: delay after the stage

```
delay_after: 5m       # wait before the next stage
```

Role. allows an external system to stabilize.

workflows: workflows in the stage (at least one)

```
workflows:
  - name: load_sales
  - name: load_stock
```

Role. lists the workflows run by the stage (workflows/<name>.yaml).

active: enables/skips a workflow (default: true)

```
- name: load_stock
  active: false
```

Role. disables a workflow without changing its definition.

delay_before: pre-start delay

```
- name: build_reports
  delay_before: 30s
```

Role. delays the start of a workflow.

timeout: time limit

```
- name: load_sales
  timeout: 20m        # exceeding limit causes failure
```

Role. prevents a workflow from blocking the thread indefinitely.

fallback: replacement workflow

```
- name: load_sales
  fallback:
    use: [load_sales_backup]          # workflows tried in order
    when: "not source_available('crm')"
    on_failure: true                  # also replaces after failure
    on_probe_error: keep              # keep | replace if condition cannot be evaluated
```

Role. switches to an alternative workflow when the condition is true.

use / when / on_failure / on_probe_error: fallback keys

See above. Role: use lists replacements; when selects fallback before start; on_failure also covers failures; on_probe_error handles conditions that cannot be evaluated.

loops: replay a segment of the thread

```
loops:
  - from: load_stock         # last workflow in segment
    to: load_sales           # first workflow in segment
    times: 3                 # total executions (1 to 1000)
    delay: 10m               # wait before each replay
```

Role. repeats a sequence (polling or batch processing).

Functions available in fallback when conditions

Function	Role
source_available('id')	the source responds
destination_available('id')	the destination responds
file_exists('path')	the file exists
file_size('path')	size, compared with 10KB, 5MB, 1GB
file_age('path')	age, compared with 30s, 5m, 24h, 7d
url_ok('url')	HTTP response 2xx/3xx
secret_defined('NAME')	the secret exists
db.load('id')	active database queries
previous.status	success · failed · skipped · inactive · none
previous.rows	rows written by the previous workflow

Example

```
fallback:
  use: [load_sales_light]
  when: "db.load('dwh') > 50 or file_size('in/orders.csv') > 1GB"
```

1.4 Sub-projects: type: project (Hydra 0.19.0+)

A step of type project runs another Hydra project: its thread.yaml, or only the workflows listed. The calling project is the master; the called projects are its sub-projects. Nothing passes implicitly except the environment.

project: the called project (required with type: project)

```
- name: crm_sync
  type: project
  project: ../crm_sync      # folder, relative to the master's root
```

Role. designates the project to run. In Hydra Studio, drop a project from the Projects section of the palette: the step is created after a cycle check.

ref: how project is read (default: path)

```
ref: path      # path: folder, portable with git (default)
ref: name      # name: project name in the Studio workspace
```

Role. chooses between a path stored in the repository and a workspace name.

env: environment of the called project (default: inherit)

```
env: inherit   # inherit | none | <environment name>, e.g. prod
```

Role. inherit applies the master's environment name with the called project's own environments/<env>.yaml, env.<env> variables and secrets.<env>; none runs without environment.

params: parameters passed to the called project

```
params:
  batch_date: "{{ param:run_date }}"   # same rank as -P batch_date=...
```

Role. gives values to the called project's parameters. A required parameter without a value stops the step before anything runs.

variables: environment variables lent for this run

```
variables:
  EXPORT_DIR: out/crm       # visible only inside the called project
```

Role. lends variables for the duration of the run; the master's own variables, .env and --env-file are not passed.

Secrets are never passed

```
variables:
  DB_PASSWORD: "${SECRET:crm_db}"   # refused: each project keeps its own secrets
```

Role. a ${SECRET:...} in params or variables is refused at validation: the called project reads its own .hydra/secrets.

workflows: workflows to run (default: the thread.yaml)

```
workflows: [ingest, report]     # run one after the other
```

Role. runs part of the called project instead of its whole thread.

outputs: parameters read back by the master

```
outputs: [last_batch]
# next steps: {{ project:crm_sync.param.last_batch }}
```

Role. lists parameters of the called project (set_param, assign_param or declared) to read once it ends. A listed output that is not published fails the step.

isolation: where the project runs (default: inprocess)

```
isolation: inprocess    # inprocess | subprocess
```

Role. inprocess runs in the same process with an isolated environment, restored afterwards; subprocess starts a separate hdrctl. Steps running in parallel always use subprocess.

timeout: maximum duration

```
timeout: 10m        # 30s, 5m, 1h or a number of seconds
```

Role. stops the called project when exceeded; the step fails.

overlap: project already running (default: wait)

```
overlap: wait      # wait | skip | fail
```

Role. a project runs once at a time (.hydra/run.lock): wait waits for the end (up to timeout), skip skips the step, fail fails it.

Results of the called project

Placeholder	Value
{{ project:STEP.status }}	success · failed · skipped
{{ project:STEP.rows }}	rows written by the called project
{{ project:STEP.duration }}	duration in seconds
{{ project:STEP.param.NAME }}	a parameter listed in outputs

```
- name: report
  type: action
  action: log
  when: "{{ project:crm_sync.status }} == 'success'"
  params: { message: "{{ project:crm_sync.rows }} rows synced" }
  depends_on: [crm_sync]
```

Role. lets the next steps of the master react to the called project.

Cycles and nesting

```
hdrctl workflow validate main     # prints the tree of called projects
#   ↳ crm_sync [ingest, report]
#      ↳ billing
# cycle: master → crm_sync → master   -> refused
```

Role. a project cannot call itself, directly or through others; more than 5 levels is refused. Checked by hdrctl run, workflow validate, Studio and the MCP tool hydra_project_calls; a project computed with {{ }} is checked at run time.

hdrctl run -w: run only some workflows

```
hdrctl run ../crm_sync -w ingest -w report --env prod
```

Role. runs the listed workflows of a project, in order, from the command line.

2. Jobs

Directory: jobs/<name>/ with sources.yaml, destinations.yaml, pipeline.yaml, transformations.yaml (optional).

2.1 Sources (sources.yaml)

sources: root

```
version: "1.0"
sources:
  orders:              # "orders" is the source ID
    type: csv
```

Role. declares at least one source that can then be referenced by ID.

type: connector (required)

```
type: postgresql   # csv, json, parquet, mysql/mariadb, postgresql/postgres, sqlserver/mssql, mongodb
```

Role. selects the reader connector; case-insensitive.

connection: connection settings

```
connection:
  host: db.local
  port: 5432
  user: etl
  password: ${SECRET:db.password}         # never hard-code secrets
  database: sales
```

Role. configures access to the source ({} for a local file). SQL keys: host, port, user, password, database, schema; SQL Server: instance, tds_version, login_timeout, timeout.

extract: what to read (required)

```
extract:
  table: orders.csv          # file, table, or...
  batch_size: 5000           # rows per batch (default 10000, minimum 10)
```

Role. defines the read operation; must contain table, query or collection.

table / query / collection

```
extract: { table: public.orders }                          # table or file
extract: { query: "SELECT * FROM orders WHERE total>0" }   # SQL only
extract: { collection: orders }                            # MongoDB
```

Role. identifies the object to read.

filter and limit: MongoDB

```
extract:
  collection: orders
  filter: { status: "paid" }        # query filter
  limit: 1000                       # maximum number of documents
```

Role. restricts the data retrieved from the database.

CSV options (in connection)

```
connection:
  base_path: ./data      # base directory for table paths (default: job directory)
  delimiter: ";"         # default ,
  quotechar: '"'
  encoding: utf-8
  lineterminator: "\n"
  strict_schema: false
```

Role. adapts reading to the file format.

schema: expected schema (MongoDB)

```
schema:
  mode: infer_strict              # auto | manual | infer_strict | hybrid
  drift_policy: warn              # fail | warn | ignore | adapt
  validation_policy: strict       # strict | best_effort | permissive
  sample_size: 100                # sampled documents
  fields:
    - name: total
      type: float                 # string, integer, float, decimal, boolean, date,
                                  # datetime, array, object, json
      path: amount.total
      required: true
      default: 0
      array_handling: flatten     # keep | flatten | explode | json
      description: "Amount"
```

Role. defines the expected document structure and response to new fields (schema drift).

2.2 Destinations (destinations.yaml)

destinations: root

```
destinations:
  dwh_orders:          # destination ID
    type: postgresql
```

Role. declares where data is written.

table / collection: target

```
load: { table: dwh.orders }      # collection is an alias for MongoDB
```

Role. specifies the destination table or file (required).

mode: write mode (default: append)

```
load:
  mode: upsert     # append | replace | upsert
  key: [order_id]  # required with upsert
```

Role. append adds rows; replace clears and writes; upsert inserts or updates using a key.

key: key columns

See above. Role: identifies a row for upsert.

batch_size: rows per write (10 to 100000; default 10000)

```
load: { batch_size: 20000 }
```

Role. balances memory consumption and throughput.

atomic: all-or-nothing writes

```
load: { atomic: true }     # rolls back the whole write on error
```

Role. avoids leaving the destination partially populated.

2.3 Pipeline (pipeline.yaml)

pipeline, from, to

```
version: "1.0"
pipeline:
  from: orders        # ID declared in sources.yaml
  to: dwh_orders      # ID declared in destinations.yaml
```

Role. links one source to one destination; transformations are applied between them.

2.4 Transformations (transformations.yaml)

```
transformations: ordered list of operations
version: "1.0"
engine: auto           # auto | pandas | duckdb (see below)
transformations:
  - select: { columns: [id, country, amount] }
  - filter: { expr: "amount > 0" }
```

Role. describes data operations executed in their listed order.

Column operations

```
select: keeps selected columns.
- select:
    columns: [id, country, amount]   # other columns are removed
rename: renames columns.
- rename:
    mapping: { cust_id: customer_id }   # old: new
cast: changes data types (int, float, str, bool, datetime).
- cast:
    mapping: { amount: float, created: datetime }
```

Row operations

```
filter: keeps rows matching the condition.
- filter: { expr: "country == 'BE' and amount >= {{ param:min_amount }}" }
sort: stable sort.
- sort: { by: [country, amount], ascending: [true, false] }
deduplicate: removes duplicates.
- deduplicate: { columns: [customer_id], keep: last }   # first | last
fill_null: fills missing values.
- fill_null: { columns: { country: "N/A", amount: 0 } }   # or value: 0 for all
trim: trims whitespace.
- trim: { columns: [name] }   # default: every text column
clean: cleans text.
- clean: { columns: [status], case: upper }   # none | lower | upper
```

text_to_rows: one row per value (Hydra 0.17.0+)

```
- text_to_rows:
    column: tags            # "a;b;c" -> 3 rows, the other columns are copied
    sep: ";"                # default ","
    regex: false            # true: sep is a regular expression, e.g. "[,;]"
    trim: true              # default true: spaces around each value removed
    keep_empty: false       # default false: empty values dropped
    index_column: tag_pos   # optional: position 1, 2, 3...
```

Role. splits a list stored in one cell. A null value, or one with no non-empty part, keeps one row with a null value: no input row is lost.

Calculation

```
calculate: creates a column.
- calculate: { column: total_ttc, expr: "amount * 1.2" }
script: restricted Python.
- script:
    inputs: [amount]
    outputs: { amount_x2: float }
    mode: vectorized         # vectorized | row
    code: |
      amount_x2 = amount * params["factor"]   # columns are variables; params = run parameters
```

Aggregation

```
aggregate: groups and aggregates.
- aggregate:
    by: [country]
    agg:
      total: { func: sum, col: amount }   # sum | count | mean | avg | min | max | first | last
      n:     { func: count, col: id }
```

Role. one row per group; columns absent from by and agg are dropped.

Window functions: window (Hydra 0.17.0+)

```
- window:
    partition_by: [customer_id]                # optional: whole table without it
    order_by: [order_date, {col: amount, desc: true}]
    columns:
      order_rank:  { func: row_number }
      prev_amount: { func: lag, col: amount, offset: 1, default: 0 }
      running:     { func: cumsum, col: amount }
      cust_total:  { func: sum, col: amount }
```

Role. adds columns and keeps every row, in its input order (unlike aggregate).

Family	Functions	Options
Rank	row_number, rank, dense_rank	none
Offset	lag, lead	col, offset (default 1), default (null)
Running	cumsum, cumcount, cummin, cummax	col (optional for cumcount)
Whole partition	sum, mean, min, max, count, first, last	col

Rank, offset and running functions need order_by. Null values sort last, even with desc; ties keep the input order. A running function skips null values (the row keeps the running value). Partition functions ignore nulls, as aggregate does; first and last return the first and last non-null value in order_by order. cumsum, cummin, cummax, sum and mean need a numeric column.

Combining data (an additional source declared in sources.yaml)

```
join: join.
- join:
    right: customers
    key: customer_id          # or left_key / right_key
    how: left                 # inner | left | right | outer | cross
merge: merge/update.
- merge: { right: corrections, key: id, delete_unmatched: false }
union: stacks rows.
- union: { right: orders_archive, distinct: true }
```

Cross join: every row × every row (Hydra 0.17.0+)

```
- join:
    right: calendar
    how: cross                # no key, left_key or right_key
    max_rows: 5000000         # default 10,000,000 rows produced
```

Role. pairs each row with each row of right (left order, then right order). The run stops before producing the batch that would exceed max_rows.

Reshaping

```
pivot: rows into columns.
- pivot: { index: [country], column: month, values: amount, aggfunc: sum }
unpivot: columns into rows.
- unpivot: { id_vars: [id], value_vars: [jan, feb], var_name: month, value_name: amount }
transpose: transposes rows and columns.
- transpose: { index_col: id, header_name: column }
engine: execution engine (at transformations.yaml root)
engine: duckdb   # auto (default) | pandas | duckdb
```

Role. auto chooses DuckDB above 2,000,000 rows for global operations (aggregate, sort, right/outer join, window). HYDRA_ENGINE overrides the file setting.

Expressions (expr, when)

Comparison, logical (and, or, not), and arithmetic operators; safe evaluation without arbitrary code execution.

3. Actions

An action is a step with type: action. Common structure:

```
- name: my_step
  type: action
  action: <name>
  params: { ... }
  undo: { ... }      # required for webhook, email, bash, powershell, ssh, python
```

log: writes a message

```
action: log
params: { message: "Loading complete" }   # default: "Step '<name>' executed"
```

Role. records execution progress.

delay: waits

```
action: delay
params: { seconds: 30 }   # or duration
```

Role. allows an external system to stabilize between steps.

condition: evaluates a condition and stores the result

```
action: condition
params:
  name: is_big                    # parameter receiving the result
  mode: expr                      # expr | structured
  expr: "{{ param:rows }} > 1000"
  # structured: left / op / right
```

Role. provides a value for the when expressions of subsequent steps.

webhook: HTTP call

```
action: webhook
params:
  url: https://hooks.example.com/notify
  method: POST                    # default POST
  headers: { Authorization: "Bearer ${SECRET:api.token}" }
  body: { status: done }          # mapping (JSON) or text
undo:
  action: webhook
  params: { url: https://hooks.example.com/cancel }
```

Role. notifies or triggers an external system.

email: sends an email

```
action: email
params:
  to: ops@example.com
  subject: "Report ready"
  body: "See attachment"
  from_addr: hydra@example.com    # default hydra@localhost
  smtp_host: smtp.example.com     # default localhost
  smtp_port: 587                  # default 1025
  username: etl
  password: ${SECRET:smtp.password}
  use_tls: true                   # STARTTLS
undo: none
```

Role. alerts the relevant recipients.

bash: shell command

```
action: bash
params:
  command: "./scripts/archive.sh"
  working_dir: ./scripts          # default current directory
  timeout: 60                     # seconds, default 60
undo: { action: bash, params: { command: "./scripts/restore.sh" } }
```

Role. executes a system command (Linux/macOS).

powershell: PowerShell command

```
action: powershell
params: { command: "Copy-Item a.csv b.csv", timeout: 60 }
undo: { action: powershell, params: { command: "Remove-Item b.csv" } }
```

Role. Windows equivalent of bash.

ssh: remote command

```
action: ssh
params:
  host: srv1.example.com
  username: etl
  command: "systemctl restart app"
  port: 22                          # default 22
  key_path: ~/.ssh/id_ed25519       # preferred
  password: ${SECRET:ssh.password}  # if no key
  timeout: 30
undo: none
```

Role. controls a remote server.

python: Python code

```
action: python
params:
  script: |                         # inline code or file_path: job.py
    print("hello")
  working_dir: .
  timeout: 60
undo: none
```

Role. implements custom logic not covered by the DSL.

set_param: creates a parameter for subsequent steps

```
action: set_param
params: { name: batch_label, value: "nightly", type: str }   # fails if already defined
```

Role. shares a value between steps ({{ param:batch_label }}).

assign_param: updates an existing parameter

```
action: assign_param
params: { name: batch_label, value: "partial" }   # fails if it does not exist
```

Role. updates an existing value during a run.

4. Parameters and secrets

4.1 Declaration

parameters (parameters.yaml file at the project root)

```yaml
version: "1.0"
parameters:
  min_amount:
    type: float           # int | float | str/string | bool/boolean | json | any (default any)
    default: 50           # value used if none is provided
    required: false       # true: fail if no value is provided
    description: "Minimum threshold"
```

Role. declares configurable project values and their types.

type / default / required / description

See above. Accepted boolean values: true 1 yes on / false 0 no off. Short form (Hydra 0.18.0+): name: value is a parameter of type any with that default.

4.2 Environments

environments/<env>.yaml

```yaml
# environments/prod.yaml
parameters:
  publish_enabled: true
  country: BE
```

Role. overrides parameters for each environment (dev, test, prod). Only parameters: variables and secrets have their own files (below).

--env / HYDRA_ENV: environment selection

```
hdrctl run . --env prod                # or HYDRA_ENV=prod
hdrctl workflow run load_sales --env prod   # Hydra 0.18.0+
```

Role. selects the environment: environments/<env>.yaml, ~/.hydra/env.<env> and .hydra/secrets.<env>. In Hydra Studio (0.18.0+), the Environment selector on the project page and in the editor header applies to every run started from Studio.

~/.hydra/env and ~/.hydra/env.<env>: global variables (Hydra 0.18.0+)

```
# ~/.hydra/env: every environment, every project of the machine
PG_HOST=db.internal
# ~/.hydra/env.prod: prod only
PG_DATABASE=shop_prod
```

Role. environment variables shared by every project of the machine, read with ${ENV:NAME}. HYDRA_HOME changes the folder (default ~/.hydra). When a run changes environment, the variables of the previous one are removed.

--env-file: variables for one run (Hydra 0.18.0+)

```
hdrctl run . --env prod --env-file extra.env   # a missing file stops the run
```

Role. adds variables above every other layer (see 4.9).

--param / -P: value for one run

```
hdrctl run . --env prod -P min_amount=100 -P country=BE
```

Role. overrides parameters for one run. Precedence: run > environment > default.

4.3 Parameter notation

{{ param:NAME }}: parameter value

```
when: "{{ param:publish_enabled }} == True"
```

Role. injects a parameter into jobs, params, when and action scripts. Unknown parameters raise an error; a standalone value retains its type.

{{ env:NAME }}: optional environment variable

```
base_path: "{{ env:DATA_DIR }}"   # no error if absent
```

Role. reads an environment variable without failing if it is missing.

${ENV:NAME}: required environment variable

```
host: ${ENV:DB_HOST}   # run stopped: "Missing environment variable: DB_HOST"
```

Role. requires an environment variable to be present.

${SECRET:NAME}: secret

```
password: ${SECRET:db.password}
```

Role. injects a sensitive value without writing it into YAML. The value is masked in everything Hydra displays (see 4.6).

4.4 Secret resolution

For ${SECRET:db.password}, in this order (the first source that knows the name wins):

#	Source	Example
1	normalized environment variable (. and - become _, uppercase)	DB_PASSWORD
2	environment variable with the exact name	db.password
3	secrets file of the active environment (Hydra 0.18.0+)	.hydra/secrets.prod
4	local secrets file, plain text or encrypted with SOPS	.hydra/secrets
5	provider declared in .hydra/secrets.yaml (Hydra 0.17.0+)	provider: sops

The environment (CI, shell, .env) always wins: production can supply every value without any file.

.hydra/secrets: local file

```
# name=value, one per line, # = comment
db.password=change-me
api.token="abc 123"
export smtp.password=s3cret
```

Role. local storage created by hdrctl init and included in .gitignore. If the file is encrypted with SOPS, Hydra detects it and decrypts it (see 4.7).

HYDRA_SECRETS_FILE / --secrets-file: alternate location

```
hdrctl run . --secrets-file /etc/hydra/secrets
```

Role. changes the secrets-file location without modifying the project. Plain or SOPS-encrypted.

.hydra/secrets.<env>: secrets of one environment (Hydra 0.18.0+)

```
# .hydra/secrets.prod (ignored by git, like .hydra/secrets)
mysql.password=Prod_Pass_456
```

Role. with --env prod, a name present here wins over .hydra/secrets; the other names still come from .hydra/secrets. Same format, plain or SOPS-encrypted.

secret_defined('NAME'): test for a secret (in when)

```
when: "secret_defined('api.token')"
```

Role. runs a step only if the secret exists.

4.5 Secrets settings (Hydra 0.17.0+)

.hydra/secrets.yaml: visibility and provider

```
# .hydra/secrets.yaml: settings only, no secret inside (safe to commit)
visibility: masked                  # plain | masked | hidden (default masked)
provider: sops                      # none (default) | sops
sops:
  file: .hydra/secrets.sops.yaml    # relative to the project root; {env} = active environment (0.18.0+)
  format: yaml                      # dotenv | yaml | json (default: from the extension)
  age_key_file: ~/keys/hydra.txt    # optional (default: SOPS_AGE_KEY_FILE)
```

Role. chooses how secrets appear in logs and where they come from. An unknown key, value or provider stops the run with an explicit error. vault, azure-keyvault and aws-secrets-manager are reserved names, not available yet.

4.6 Secret visibility in logs (Hydra 0.17.0+)

Mode	What is displayed	Use
plain	the value, in clear; a warning reminds it	debugging only
masked	[MASKED] in place of the value	default
hidden	the line is omitted and counted: [1 line(s) hidden: they contain a secret]	strict audits, shared logs

--secrets-visibility / HYDRA_SECRETS_VISIBILITY: per run

```
hdrctl run . --secrets-visibility hidden
hdrctl workflow run load_sales --secrets-visibility plain   # or HYDRA_SECRETS_VISIBILITY=plain
```

Role. overrides the project setting for one run. Precedence: --secrets-visibility > HYDRA_SECRETS_VISIBILITY > visibility: > masked.

Example

```
- name: notify
  type: action
  action: python
  params:
    script: "print('token=${SECRET:api.token}')"
# masked  ->  token=[MASKED]
# hidden  ->  [1 line(s) hidden: they contain a secret]
```

Role. the script still receives the real value; only what is displayed or kept changes.

What is masked

Logs (every logger, step logs, API, CLI output), error messages and tracebacks, the output of bash, powershell, python and ssh actions, webhook and email log lines, the rollback (undo) report and the runs stored by the API.

Masked forms: the raw value, its URL-encoded form (connection strings), its JSON-escaped form, and each line of a multi-line value (PEM keys). Every value of the secrets file is masked, even if no ${SECRET:} uses it.

Rules: a value shorter than 4 characters is not masked and a warning names it; when two settings disagree for the same value, the strictest wins. Values sent to connectors, scripts and webhooks are never modified.

4.7 Encrypted secrets with SOPS and age (Hydra 0.17.0+)

Create the key and encrypt the file

```
age-keygen -o ~/.config/sops/age/keys.txt        # prints the public key age1...
sops --encrypt --age age1... secrets.yaml > .hydra/secrets.sops.yaml
sops .hydra/secrets.sops.yaml                    # edit: decrypted in the editor only
```

Role. SOPS encrypts the values, not the keys: the file can be committed and reviewed. Install sops and age first (Windows: winget install Mozilla.SOPS FiloSottile.age).

secrets.yaml before encryption

```
db:
  password: change-me     # read as ${SECRET:db.password}
api:
  token: abc123           # read as ${SECRET:api.token}
```

Role. nested keys are joined with dots. Never commit this plain file: delete it once encrypted.

Behaviour

The file is decrypted in memory on the first secret requested; the clear text never touches the disk. sops chooses the key (age, PGP, cloud KMS); with age, it reads SOPS_AGE_KEY_FILE or sops.age_key_file. HYDRA_SOPS_BIN gives the path of sops when it is not on PATH.

Hydra Studio refuses to edit an encrypted file (edit it with sops). Explicit errors: "sops is not installed or not on PATH", "cannot decrypt ... no age key can open this file".

4.8 Passing values to scripts

Where	How the parameter is provided
bash, powershell, ssh, python	environment variable named after the parameter: $country, $env:country, os.environ["country"]
transformation script	read-only mapping params["min_amount"]

Secrets reach a script only through ${SECRET:NAME} written in its params.

Known limitations: a secret that a script reads directly from the environment, without ${SECRET:} or the secrets file, is not masked; nor is a transformed value (base64, split).

4.9 Order of the environment variables

#	Layer (the last one wins)	Since
1	~/.hydra/env (every environment)	0.18.0
2	~/.hydra/env.<env> (active environment)	0.18.0
3	.env at the project root (folder containing .hydra/)	
4	.env of the job folder	
5	--env-file <file> / HYDRA_ENV_FILE	0.18.0

A file wins over a variable already set in the system. hdrctl run jobs/<job> also reads the .env at the project root (0.18.0+).

4.10 Managing them: CLI and Studio (Hydra 0.18.0+)

What	CLI	Hydra Studio
environments, global variables	hdrctl env list | show | create | rename | delete | set | unset	Global settings > Environments
parameters	hdrctl param list | set | unset (--env, --type, --required)	Project settings > Parameters (one column per environment)
secrets (names only)	hdrctl secret list | set | delete (--env, --stdin)	Project settings > Secrets (common or per environment)

```
hdrctl env create prod
hdrctl env set PG_DATABASE shop_prod --env prod
hdrctl param set batch_size 5000 --env prod
hdrctl secret set mysql.password --env prod     # typed hidden, never an argument
hdrctl env delete qa -y                         # files copied first to ~/.hydra/_backups
```

Role. writes the files of 4.1 to 4.9. A secret value is never displayed, by the CLI, Studio, the API or the MCP server.